

I2C OLED Display Experiment

I2C OLED Display Experiment

- 1. I2C OLED Display Overview
- 2. Core Concepts
 - 2.1 I2C Bus Basics
 - 2.2 SSD1306 Control Byte
 - 2.3 Page Addressing Mode and Font Rendering
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code
 - 5.1 I2C Bus Initialization
 - 5.2 I2C Low-Level Driver
 - 5.3 SSD1306 Initialization Sequence
 - 5.4 Transpose Rendering Engine
 - 5.5 Font Selection and String Rendering
 - 5.6 `setup()` — Three-Line Startup
- 6. Operation Flow
 - 6.1 Compile and Flash
 - 6.2 Normal Phenomena
 - 6.3 Verification Methods
- 7. Common Issues

1. I2C OLED Display Overview

I2C is a two-wire serial bus (SDA+SCL), with multiple devices connectable on a single bus. This experiment drives an SSD1306 OLED screen (128×32 pixels) via I2C to display the string "YAHBOOM". The code does not depend on any third-party graphics library, using pure handwritten I2C low-level driver + font rendering engine, demonstrating the complete process from I2C bus operation to screen pixel output.

Typical Application Scenarios:

Application Type	Examples
Information Display	Real-time data such as temperature/humidity, voltage, time, IP address
Sensor Peripherals	BME280, MPU6050, BH1750, RTC clock
Human-Machine Interface	OLED menu, parameter configuration interface
Multi-device Interconnection	Multiple I2C devices with different addresses on the same bus

2. Core Concepts

2.1 I2C Bus Basics

I2C requires only two wires: SCL (clock) and SDA (data), both generated by the master. Each slave has a 7-bit address, and the master sends the address first after START to select the target slave.

```
Master Write: START → Device address(7bit)+W(1bit) → ACK(1bit) → Register address(8bit) →
ACK(1bit) → Data(8bit) → ACK(1bit) → STOP
Master Read: START → Device address(7bit)+W(1bit) → ACK(1bit) → Register address(8bit) →
ACK(1bit) →
                RESTART → Device address(7bit)+R(1bit) → ACK(1bit) → Data(8bit) → NAK(1bit) → STOP
```

In this experiment, the I2C address is 0x3C (SSD1306 default), and SCL frequency is 100kHz (standard mode).

2.2 SSD1306 Control Byte

When SSD1306 communicates via I2C, each frame of data must be preceded by a **Control Byte** to distinguish commands from data:

Control Byte	Meaning
0x00	Subsequent bytes are commands (configure display parameters)
0x40	Subsequent bytes are GDDRAM data (pixel bitmap)

2.3 Page Addressing Mode and Font Rendering

SSD1306 128×32 pixels = 4 pages × 128 columns (each page is 8 pixels high). Font source data is **row-major** (one uint16_t per row), but SSD1306 requires **column-major** writing (8 vertical pixels at a time). Therefore, "transpose rendering" is needed — for each column, extract pixels from multiple rows of font source data and assemble them into an 8-bit vertical byte.

3. Hardware Dependencies

GPIO Pins:

Pin	Peripheral	Configuration	Description
GPIO41	SDA	I2C data line	Connected to SSD1306 SDA pin
GPIO42	SCL	I2C clock line	Connected to SSD1306 SCL pin

ESP32-S3 I2C Hardware Features:

Feature	Parameter
Number of I2C controllers	2 (I2C0 / I2C1)

Feature	Parameter
Supported modes	Master / Slave
Speed grades	Standard 100kHz / Fast 400kHz / High-speed 1MHz
GPIO mapping	Any GPIO can be configured as SDA/SCL via IO MUX
Slave address	7-bit or 10-bit (7-bit <code>0x3C</code> in this experiment)
FIFO depth	32 bytes (independent TX/RX)

OLED resolution is 128×32. Depends on `oled_fonts.h` (embedded 3 sets of font data).

4. Project Architecture and Flow

```

setup()
├ wire.begin(SDA, SCL) + setClock(100kHz) // Initialize I2C
├ ssd1306_init()                          // SSD1306 initialization sequence
│   ├── clock/mux ratio/offset/charge pump series of commands
│   ├── oled_clear() clear screen
│   └── 0xAF enable display
└ oled_show_str(0, "YAHBOOM")             // Display string at column 0

oled_show_str(x, str)
└ for each char:
    └ select corresponding rendering function based on font:
        oled_show_11x18 / 7x10 / 16x26
        └ render_page(): Column-major transpose → oled_data()

```

5. Source Code

Source Code Location: `Source code` → `Arduino` → `2.Serial communication` → `2.IIC_OLED` → `2.IIC_OLED.ino`

5.1 I2C Bus Initialization

Set I2C pins and clock frequency. ESP32 supports GPIO matrix remapping, with SDA/SCL allocatable to any pin.

```

void oled_bus_init(int sda, int scl) {
    wire.begin(sda, scl);
    wire.setClock(100000); // 100kHz standard mode, SSD1306 max 400kHz
    delay(100);           // wait for bus and device power-up stabilization
}

```

`wire.begin(sda, scl)` must be called before all I2C communication. `100000` = 100kHz is a conservative value, improving reliability in long wire or breadboard scenarios.

5.2 I2C Low-Level Driver

`oled_cmd` and `oled_data` send commands and data respectively — the only difference is the Control Byte.

```
void oled_cmd(uint8_t c) {
    wire.beginTransaction(OLED_ADDR);
    wire.write(0x00); // Control Byte = 0x00 → subsequent bytes are commands
    wire.write(c);
    wire.endTransmission();
}

void oled_data(uint8_t d) {
    wire.beginTransaction(OLED_ADDR);
    wire.write(0x40); // Control Byte = 0x40 → subsequent bytes are GDDRAM data
    wire.write(d);
    wire.endTransmission();
}
```

5.3 SSD1306 Initialization Sequence

After SSD1306 powers on, it needs to be configured item by item according to the datasheet command sequence.

```
void ssd1306_init(void) {
    oled_cmd(0xAE); // Turn off display first
    oled_cmd(0xD5); oled_cmd(0x80); // Clock division
    oled_cmd(0xA8); oled_cmd(0x1F); // Multiplex ratio = 32 rows
    oled_cmd(0x8D); oled_cmd(0x14); // Charge pump enable ← Critical! forgetting this
    results in no display
    oled_cmd(0x20); oled_cmd(0x02); // Page addressing mode
    oled_cmd(0xA1); // Segment remapping
    oled_cmd(0xDA); oled_cmd(0x12); // COM pin configuration
    oled_cmd(0x81); oled_cmd(0xCF); // Contrast 207/255
    oled_clear();
    oled_cmd(0xAF); // Enable display
}
```

The most easily overlooked is `0x8D/0x14` (charge pump enable) — without it, the SSD1306 internal high-voltage driver doesn't work, resulting in a fully black screen.

5.4 Transpose Rendering Engine

Font source data is row-major, SSD1306 needs column-major. `render_page()` assembles 8 rows of source data into one vertical byte per column.

```
static void render_page(uint8_t x, const uint16_t *data, int w,
                        int sr, int n, int pg) {
    oled_set_pos(pg, x);           // Position to target page and column
    for (int c = 0; c < w; c++) { // Iterate through each column
        uint8_t b = 0;
        for (int r = 0; r < n; r++) // Iterate through n rows covered by this page
            if (GLYPH_BIT(data, sr + r, c))
                b |= (1 << (n - 1 - r));
        oled_data(b);              // Write 8 vertical pixels
    }
}
```

The `GLYPH_BIT` macro extracts the pixel at row, col from row-major source data. Source data is `uint16_t`, with bit[15] corresponding to the leftmost pixel.

5.5 Font Selection and String Rendering

Switch fonts via the global pointer `g_font`. `oled_show_char` automatically dispatches to the corresponding rendering function based on the current font.

```
static FontDef_t *g_font = &Font_11x18; // Default 11x18 font

void oled_set_font(uint8_t font_id) {
    g_font = oled_font_select(font_id); // Available: 7x10 / 11x18 / 16x26
}

void oled_show_char(uint8_t x, char ch) {
    if (g_font == &Font_7x10)      oled_show_7x10(x, ch);
    else if (g_font == &Font_11x18) oled_show_11x18(x, ch);
    else if (g_font == &Font_16x26) oled_show_16x26(x, ch);
}

void oled_show_str(uint8_t x, const char *str) {
    uint8_t w = g_font->FontWidth; // Get current font width
    while (*str) {
        oled_show_char(x, *str);    // Render one character
        x += w;                     // Advance one character width
        str++;
    }
}
```

Calling `oled_set_font(OLED_FONT_16X26)` switches to the extra-large font. All subsequent `oled_show_str` calls automatically use the new font size.

5.6 `setup()` — Three-Line Startup

```
void setup() {
  oled_bus_init(SDA_PIN, SCL_PIN);    // ① Configure I2C pins and clock
  ssd1306_init();                     // ② Send SSD1306 initialization command sequence
  oled_show_str(0, "YAHBOOM");         // ③ Display string at column 0 with default font
}

void loop() { delay(1000); }
```

6. Operation Flow

6.1 Compile and Flash

Arduino Flash Process: See 1. Before Development → 2. Build and Flash.

- 1. Ensure `oled_fonts.h` and `oled_fonts.cpp` are in the same directory as `.ino`
- 2. Open `2.IIC_OLED.ino`

6.2 Normal Phenomena

After power-on, the OLED screen displays "YAHBOOM" (default 11×18 font), static without movement.



6.3 Verification Methods

Operation	Expected Result
Observe OLED after power-on	Displays "YAHBOOM"
Change font size (<code>oled_set_font(OLED_FONT_16X26)</code>)	Font becomes larger
Change I2C address to <code>0x3D</code>	No display (address mismatch)

Font after size modification.



7. Common Issues

Phenomenon	Cause	Solution
Screen fully black, no light	Charge pump not enabled	Confirm <code>0x8D / 0x14</code> is in initialization sequence
Screen lights up but no content	I2C address error	SSD1306 SA0 pin determines address: <code>0x3C</code> or <code>0x3D</code>
Display garbled/corrupted	I2C timing unstable	Lower SCL frequency to 100kHz
Compilation error <code>Font_11x18</code> undefined	<code>oled_fonts.h</code> path incorrect	Confirm file is in same directory as <code>.ino</code>